RoboSub Virtual Environment

Version 1.0

Author: Abhishek Shankar (abshanka@mathworks.com)

Contents

Introduction

The RoboSub Virtual Environment is a simulation environment that mimics the RoboSub competition held by RoboNation. It consists of a pool with dimensions 25m x 50m x 2.25m with the course components such as gates, buoys, torpedo target boards, etc. It allows users to add their autonomous underwater vehicle (AUV) and provide the necessary details to simulate the movement of their AUV in the pool environment. Further, it also simulates sensor readings of an IMU, DVL and camera.

The document explains the details in the model and briefly touches on the visualization environment. The systems requirements and the steps to install and run the simulation are given in the following sections. More details on the Simulink model and the environment can be found in later sections.

System Requirements and Product Dependencies

To run the simulation, the following system requirements must be met

- OS: Windows 11/Ubuntu 22.0 (or later)
- RAM: Minimum of 8GB
- GPU: Nvidia 1080 or higher with at least 8GB of VRAM
- Disk Space: ~2GB

Product Dependencies:

- MATLAB 2025b
    - Simulink
    - Simulink 3D Animation
    - Aerospace Blockset
    - ROS toolbox
    - Robotics System Toolbox
    - Sensor Fusion and Tracking Toolbox

Note: Please use the right installation method based on your OS. If you do not have a license, you can request a free license (if you are RoboSub participant) here. You can also use your university license if available. If you are not participating in the competition but would like to get the license for research/academic purposes, please contact us at roboticsarena@mathworks.com and we will review it on a case-by-case basis.

Getting Started

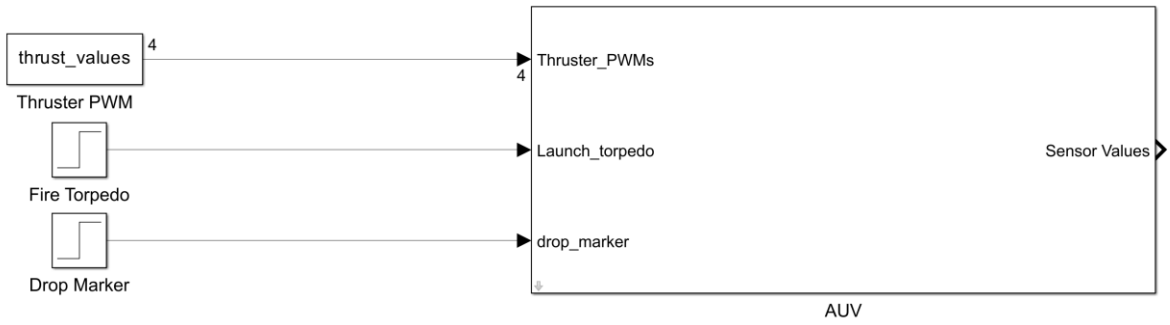
To get started with the simulation:

1. Clone the Virtual RoboSub repo into your local machine and download the Unreal executable file from the attached link and extract the contents to a local folder.
2. Open MATLAB 2025b and navigate to the Virtual RoboSub repo from step 1
3. Open the RoboSub.prj project. This will load the necessary files, update the folders in path and open the Simulink model
4. Once the model opens, go under the AUV masked subsystem. This can be done by right clicking on the AUV block -> Mask -> Look Under mask. Or you can click on the AUV and press 'ctrl + U'
5. Navigate to the 'Visualization' subsystem block and open 'Simulation 3D Scene Configuration'. The first time you open it, it might throw an error stating that the specified file cannot be found. Click 'OK', and you will see the dialog box of the 3D Scene configuration block.
6. Click on 'Browse' next to the Project name and browse to the location of the Unreal executable that you extracted in Step 1
7. Click apply and OK to close the window. Navigate to the model top level and click on play on the tool strip (under the Simulation tab). This should run the model with the 3D visualization

Note: If you face any errors or unexpected behaviors, please contact us at roboticsarena@mathworks.com or raise an Issue in the GitHub link.

Simulink Model

The heart of the environment is the Simulink model 'AUV.slx'. The model on the top level contains a masked subsystem which takes in thruster PWM's, torpedo and marker launch signals as input and outputs the sensor readings from the simulated AUV.



The model has a sample time of 10ms and the runtime for the simulation is set to Infinity (Inf) which means that the user is responsible for deciding when to stop the simulation.

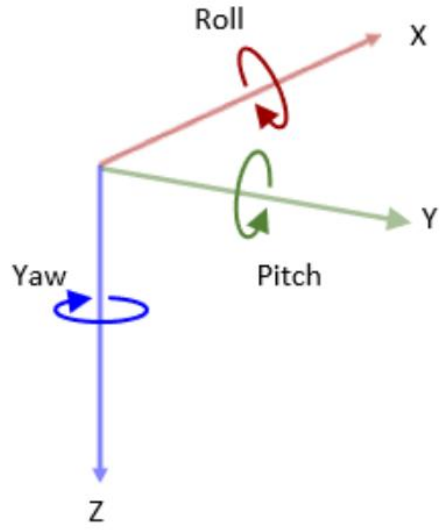
Under the mask, the AUV block is divided into four subsystems:

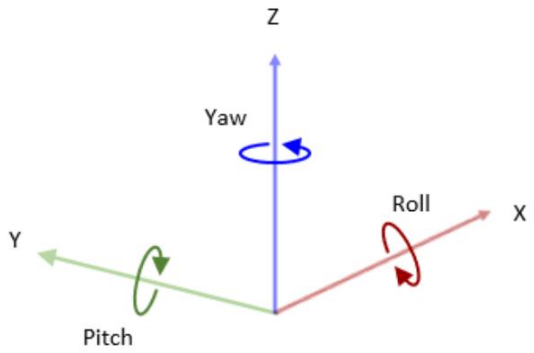
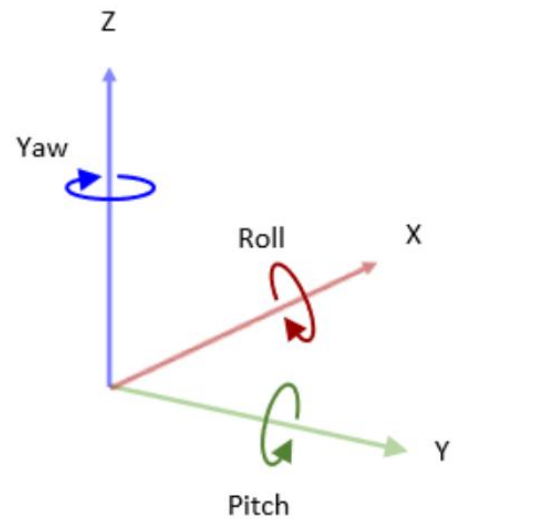
- AUV Dynamics
- Sensors
- Actuators
- Visualization

Each block simulates the respective subsystem of the AUV and the execution priority is automatically assigned by Simulink based on the required inputs for each subsystem. It is not recommended to change the execution order.

Coordinate Systems

The model uses the SAE J670 coordinate system whereas MATLAB by default uses the right-handed cartesian coordinate system with z-axis pointing up. The World in Unreal Engine uses the Unreal Coordinate system which is left-handed Z up coordinates. All the coordinates are explained in the table below

Coordinate System Description	Visual Representation
The SAE J670 standard coordinate system is a right-handed Cartesian coordinate system with Z-down orientation, in m and rad. This is defined in the SAE J670 (add reference) standard. This coordinate system is used for aerospace applications. For more information, see Body Coordinates (Aerospace Blockset). Axes • X-axis points away from the viewer. • Y-axis points to the right. • Z-axis points downward. Rotation • Roll — Clockwise rotation about X-axis • Pitch — Clockwise rotation about Y-axis • Yaw — Clockwise rotation about Z-axis	

The MATLAB coordinate system is a right-handed Cartesian coordinate system with *Z*-up orientation, in m and rad. Axes • *X*-axis points away from the viewer. • *Y*-axis points to the left. • *Z*-axis points upward. Rotation • Roll — Clockwise rotation about *X*-axis • Pitch — Clockwise rotation about *Y*-axis • Yaw — Clockwise rotation about *Z*-axis	
The world uses the Unreal Engine® coordinate system. The Unreal Engine coordinate system is a left-handed Cartesian coordinate system, in m and rad. Axes • *X*-axis points away from the viewer. • *Y*-axis points to the right. • *Z*-axis points upward. Rotation • Roll — Clockwise rotation about *X*-axis • Pitch — Clockwise rotation about *Y*-axis • Yaw — Counterclockwise rotation about *Z*-axis	

AUV Mask

The top level AUV mask is where the user is required to input all the information necessary for the simulation. The mask has four tabs as shown below

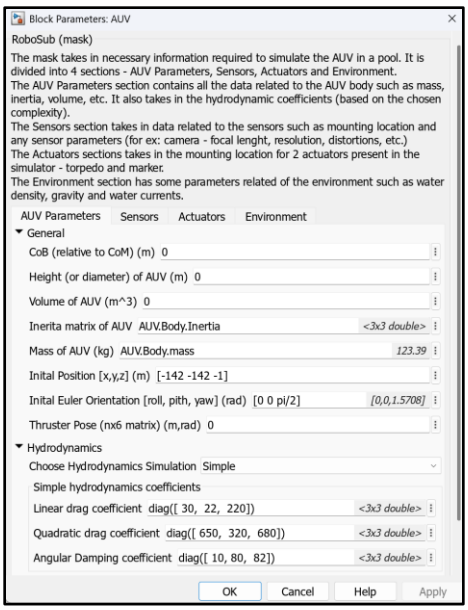


Figure 1 AUV Mask

Inputs and Outputs

The masked block has 3 inputs

- Thruster PWM – This is a nx1 vector of pwm values, where n is the number of thrusters in the AUV. Note that the order of pwm values should be the same as the thruster pose order as described in the AUV parameters.
- Lauch Torpedo – This input expects a step signal that determines when to launch the torpedo.
- Drop Marker – This input expects a step signal that determines when to drop the marker.

The block outputs sensor values as a bus with the following signals

- Acceleration values
- Angular speed
- Magnetic field
- Depth (DVL)
- Linear speed (DVL)
- DVL reference
- Image from forward facing camera
- Depth from forward facing camera
- Image from downward facing camera
- Depth from downward facing camera

AUV Parameters

This tab collects AUV body and position data along with the hydrodynamic coefficients. The user can choose between a 'simple' or 'complex' hydrodynamic simulation based on the level of detail available for the AUV. We recommend starting with a simple simulation and moving to more complex simulation as the AUV matures and more experimental data is available. The difference in these simulations will be explained under the Hydrodynamics section under AUV Dynamics.

The 'General' parameters require the following information. Their sizes, units and data types are listed in the table below

Parameter Name	Units	Dimension	Data Type
Path to Robot model	NA	NA	Character array
Mass of AUV	Kg	1x1	Double
Volume of AUV	m^3	1x1	Double
Inertia Matrix of AUV		3x3	Double
Center of Buoyancy (with respect to Center of Mass)	m	[1x3]	Double
Height (or diameter of AUV)	m	1x1	Double
Initial Position [x,y,z]	m	[1x3]	Double
Initial Orientation [roll, pitch, yaw]	rad	[1x3]	Double
Thruster Pose (nx6 matrix of thrusters where n corresponds to number of thrusters. Each row contains [x y z yaw pitch roll] of the thruster relative to the center of mass)	m, rad	[nx6]	Double

The hydrodynamic coefficients for the auv and the associated information is given in the table below:

Hydrodynamics Simulation	Parameter Name	Description	Dimension	Data Type
Simple	Linear Drag Coefficient	This is the drag coefficient that dominates during slow speeds (<1m/s). Usually lower along the x-axis (body axes).	[3x3] diagonal matrix	Double
	Quadratic Drag Coefficient	This is the drag coefficient that dominates during	[3x3] diagonal matrix	Double

		high speeds. Usually lower along the x-axis (body axes).		
	Angular Drag Coefficient	Angular drag is modelled as a linear acting force. For a cylindrical body, it is lower along x axis.	[3x3] diagonal matrix	Double
Complex	Angular Drag Coefficient	Angular drag is modelled as a linear acting force. For a cylindrical body, it is lower along x axis.	[3x3] diagonal matrix	Double
	CX			
	CY			
	CZ			
	CMM			
	CMN			
	CMK			
	Alpha_range			
	Beta_range			

Sensors

The sensors tab allows the user to add information related to the available sensors on the AUV such as cameras, IMU, DVL, etc.

The camera blocks are taken from the sensor block from Robotics System Toolbox. The MathWorks documentation page provides all the necessary information on these blocks and how to customize them.

The IMU is a Simulink block from the Sensor Fusion and tracking toolbox and this is extensively documented on the MathWorks website as well.

The Doppler Velocity Logger (DVL) is modelled using a simple MATLAB script and the user must input the mounting location and the tracking mode of the DVL.

Actuators

The model currently supports two actuator systems – a torpedo and a marker. The user only inputs the location of the actuators. Other properties such as mass, drag coefficients and size are fixed. While the model only supports a single marker and torpedo, the blocks can be duplicated to have more of these. The details on the block will be discussed in later chapters.

Environment

The environment tab consists of some constant such as water density and acceleration due to gravity. While these are populated with default values, it can be changed to simulate a different environment if required. For example, the density can be changed to reflect the density of salt water (for ocean simulation) or a different fluid.

Another parameter is water current which is a 1x3 row vector with speeds (m/s) in x and y axis, the current in z is assumed 0. The user has three options here – no current, random current and custom value for water current. This lets the user set their preference to test their control and navigation algorithms under varying conditions.

AUV Dynamics

The dynamics of the AUV is calculated by getting total forces and moments acting on the AUV's center of mass (in body axes) and passing it to the 6DOF Quaternion block to get the vehicle states such as speed, acceleration, orientation, position, etc. The forces and moments are produced by the environment and the thrusters.

Environmental Forces and Moments

The forces due to the environment are gravity, buoyancy and hydrodynamic forces and moments.

Hydrostatics

The model assumes the center of body (CoBd) is the same as center of mass (CoM) so gravity does not produce any moment. The center of buoyancy (CoB) is a location specified by the user and this is used to calculate the moment due to buoyancy. The buoyancy force is calculated based on the equation $F_B = \rho.V.g$ where ρ is the density of water, V is the volume of the AUV and g is the acceleration due to gravity. Volume of the AUV is specified by the user and the fraction of the vehicle submerged is calculated based on the z-position of the AUV.

Hydrodynamics

The hydrodynamic forces and moments are calculated in two ways (determined by the user) and is implemented using a variant subsystem.

Simple Hydrodynamics

The simple model for hydrodynamics models the forces and moments as a mix of linear and quadratic model as shown below.

The linear drag coefficient acts at slow speeds (< 1m/s) and the quadratic drag coefficients acts at higher speeds. While this is not accurate, it provides a decent approximation of the

drag forces that can be used as a starting point. The linear coefficients can be estimated in several ways. For the drag in z-axis, a quick way to estimate the value is to place the AUV at a known depth and note the time it takes to surface again (for a positively buoyant AUV. Consider the opposite for a negatively buoyant system). Something similar can be performed for the x and y axes as well where an initial speed is provided and the time it takes to come to a stop is noted. These measurements can be used to roughly estimate the linear coefficient so that the model matches the real-world data. A similar process can be used to estimate the quadratic coefficients as well, but care should be taken to set the initial speed greater than 1 m/s.

The angular drag coefficients are modelled the same way in simple and complex subsystems. It is a linear opposing force the angular velocity of the AUV body. These coefficients can also be estimated by performing simple experiments such as providing a small initial angular speed and noting the time it takes for the AUV to settle.

Note: All the coefficients are 3x3 diagonal matrices of datatype double.

Complex Hydrodynamics

This subsystem considers that drag coefficients may change with angle of attack, side slip, and AUV area. The side slip and angle of attack is calculated using Incidence, Sideslip & Airspeed block. The aerodynamic forces and moments block is used to calculate the hydrodynamic forces. The inputs used for the block are the coefficients in the body axes (axial force C_x, side force C_y, normal force C_z, rolling moment C_l, pitching moment C_m, yawing moment C_n). These coefficients are provided from a lookup table which contains data either derived experimentally or through comprehensive CFD analysis.

Thruster Forces and Moments

The forces and moments from the thrusters are calculated by taking in the position and orientation of the thruster and the PWM signal. The PWM signals are converted into thrust values using a transfer function for T200 thrusters. The transfer function details and how it was derived can be found in the linked video. These calculations are done in a For Each Subsystem which repeats the calculations for each thruster on the AUV. The thrust and moment calculations is shown in the image below. The thrust values act in x-axis of the thruster which uses the body coordinate system.

Note: The thruster poses and PWM's should be entered in the same order in the mask

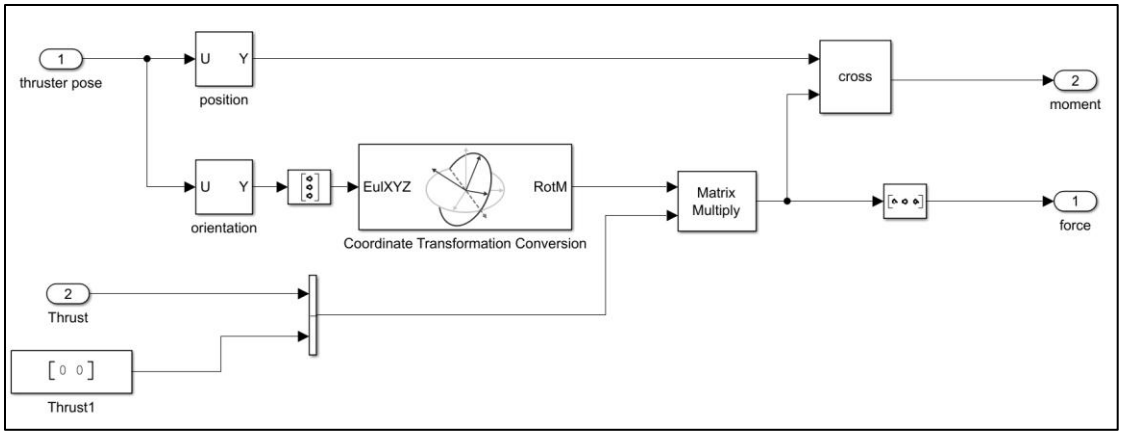


Figure 2 Force and moment calculation from thrusters

Sensors

The model provides several sensors such as DVL, pressure sensor, IMU and cameras. The parameters for these sensors are set in the top level AUV mask. The IMU is a Simulink block that comes with the sensor fusion and tracking toolbox and the camera block is included in the robotics system toolbox.

The DVL simulation produces a noisy measurement of the vehicle's linear velocity in body coordinates, optionally including bias drift, white measurement noise, random dropouts, and altitude estimation.

It can operate in two modes:

- Bottom-track: measures velocity relative to the seafloor (true ground velocity).
- Water-track: measures velocity relative to the moving water mass (adds current effect).

The model is lightweight but statistically realistic for use in EKF-based navigation and controller validation. The mathematical model is explained below.

Relative Velocity

Let:

- $V_b = [u \quad v \quad w]^T$: vehicle linear velocity in body frame
- V_c^n: current velocity in NED (Earth) frame
- $R_n^b(\phi, \theta, \psi)$: rotation matrix from NED → body

Then the water-relative velocity in the body frame is:

$$V_{rel}^{b} = V_b - R_n^b V_c^n$$

The DVL measures this vector (depending on mode):

$$V_{DVL}^{b} = \begin{cases} V_{rel}^{b} \mid water - track \\ V_b \mid bottom - track \end{cases}$$

Measurement Noise Model

Each axis includes:

$$V_{meas} = V_{DVL}^{b} + b(t) + \epsilon$$

Where,

- $b(t)$: slowly varying bias modeled as a 1st-order Gauss–Markov process
- $\epsilon \sim \mathcal{N}(0, \sum_v)$: white Gaussian noise
- Occasional dropouts simulate lost bottom-lock or bad correlation.

Altitude

Altitude is optionally estimated from the difference between vehicle depth and known seafloor depth:

$$h = \max\left(z_{seafloor} - z_{vehicle}, 0\right)$$

Actuators

The model provides two actuating systems: Torpedo and Marker Dropper as shown below

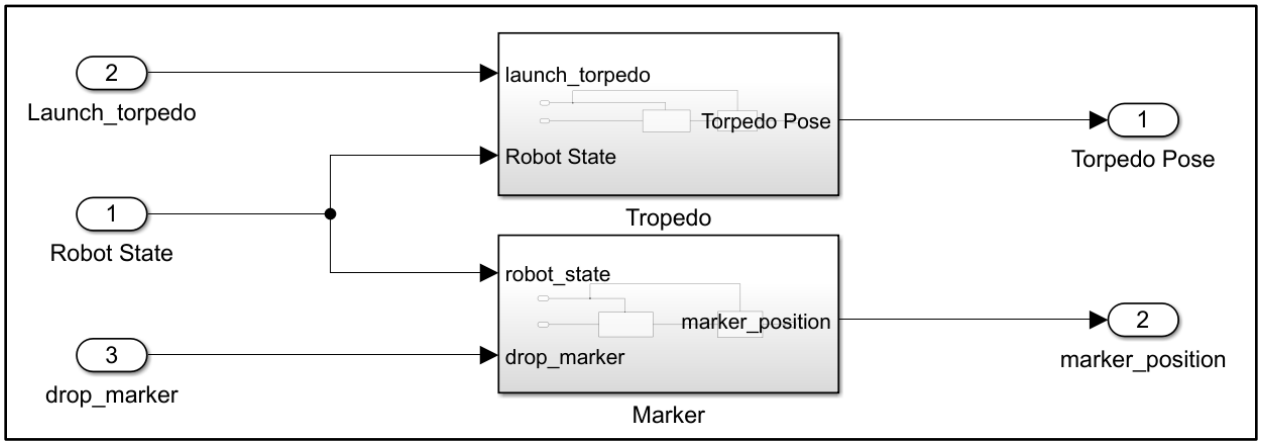


Figure 3 Actuators on the AUV

Torpedo

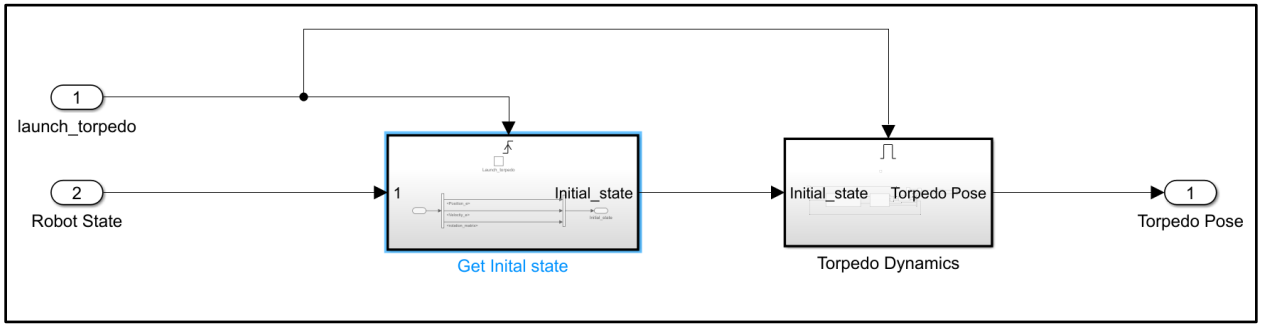


Figure 4 Torpedo Subsystem

The torpedo subsystem calculates the dynamics of the torpedo and outputs the position and orientation relative to the AUV body. First, the AUV's position and orientation at the time of firing is obtained using a triggered subsystem. The torpedo dynamics is then calculated similar to the AUV dynamics where the starting location and orientation is obtained from the AUV state with the offset of the torpedo position on the AUV body. The system currently does not consider the AUV's velocity. This is a limitation and will be addressed in the future.

Marker

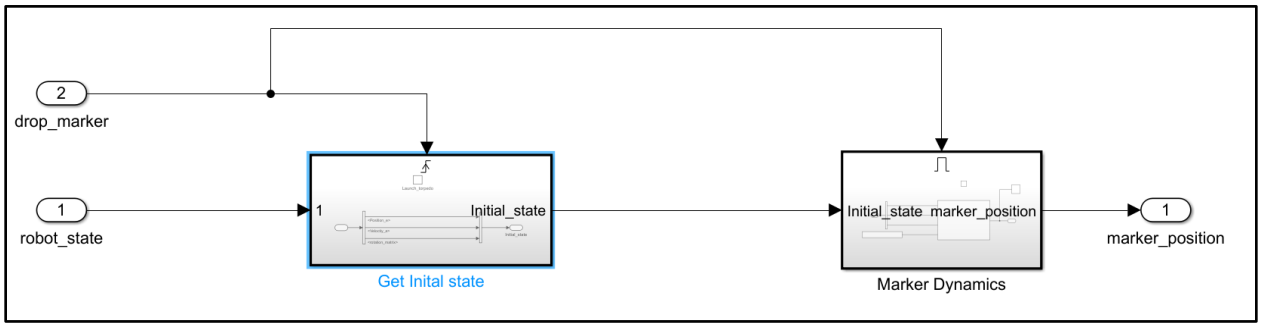


Figure 5 Marker Subsystem

The marker subsystem is similar to the torpedo system where it takes the AUV's position and orientation while dropping the marker to calculate the marker's movement. In addition to the AUV pose, it also takes in the AUV's velocity to determine the motion of the marker. The motion is calculated using Newton's equation of motion as described below

$$p_t^n = p_{t-1}^n + u_t^n . t + \frac{1}{2} a t^2$$

Where,

p_t^n is the position of the marker at time t

p_{t-1}^n is the position of the marker in the previous timestep

$u_t^n = [u \quad v \quad w]^T$ is the velocity of the AUV in inertial axes at time t

$a^n = [0\ 0\ g]^T$ is the acceleration of the AUV in inertial axes. The acceleration in x and y axes is assumed to be 0.

The velocity is updated with the following equation:

$$u_t^n = u_{t-1}^n + at - C_{drag}.u_{t-1}^n$$

C_{drag} is the coefficient of drag for the marker. It is a 3x3 diagonal matrix with value 0.02.

Visualization

The visualization subsystem contains the Simulation 3D scene configuration block and Simulation 3D Actor blocks for the various actors (AUV, marker, torpedo, etc.) in the scene. The scene configuration block tells the model to either run an inbuilt scene, co simulate with Unreal Engine or use a compiled executable. For the RoboSub virtual environment, choose Unreal Executable as the scene source and browse to the executable location on your PC for the project name. The scene should be set to: Game/Maps/RoboSub_pool.

There is an option to hide the simulation window, and this is through a check box at the bottom of the mask. This helps when the user does not need to see the simulation but still gets camera images. This also reduces the computation required as the scene does not have to be rendered fully. The environment is built with Unreal Engine 5.3 and uses an experimental water plugin and custom shaders to render the underwater look.